

Ashwath Raj M | AP25110010712 | Core CSE A

GITHUB LINK: <https://github.com/Ashwath-Raj/CSE-204-DAA-assignments->

1. Converting Recursive Programs to Non-Recursive Programs

Towers of Hanoi Problem

Method a — Implement Using Recursion

Problem

Implement the Towers of Hanoi problem using recursion.

Method

The Towers of Hanoi problem is solved recursively by moving the required disks between the source, auxiliary, and destination rods.

Out Put:

```
19      int n;

Problems Output Debug Console Terminal Ports
• ~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 3/Source-Code cd "/home/ashwath-raj/Academics/3.Semester3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3_ TowerOfHanoiLLstackandqueue/Source-Code/" && g++ 1.aTowersofHanoiProblemRecusion.cpp -o 1.aTowersofHanoiProblemRecusion && "/home/ashwath-raj/Academics/3.Semester3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3_ TowerOfHanoiLLstackandqueue/Source-Code/"1.aTowersofHanoiProblemRecusion

TOWERS OF HANOI USING RECURSION
=====
Enter Number of Disks: -1
Invalid Number of Disks!!
```

```
Problems Output Debug Console Terminal Ports
cd "/home/ashwath-raj/Academics/3.Semester3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3_ TowerOfHanoiLLstackandqueue/Source-Code/" && g++ 1.aTowersofHanoiProblemRecusion.cpp -o 1.aTowersofHanoiProblemRecusion && "/home/ashwath-raj/Academics/3.Semester3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3_ TowerOfHanoiLLstackandqueue/Source-Code/"1.aTowersofHanoiProblemRecusion
• ~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 3/Source-Code cd "/home/ashwath-raj/Academics/3.Semester3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3_ TowerOfHanoiLLstackandqueue/Source-Code/" && g++ 1.aTowersofHanoiProblemRecusion.cpp -o 1.aTowersofHanoiProblemRecusion && "/home/ashwath-raj/Academics/3.Semester3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3_ TowerOfHanoiLLstackandqueue/Source-Code/"1.aTowersofHanoiProblemRecusion

TOWERS OF HANOI USING RECURSION
=====
Enter Number of Disks: 4
Move Disk 1 from A to B
Move Disk 2 from A to C
Move Disk 1 from B to C
Move Disk 3 from A to B
Move Disk 1 from C to A
Move Disk 2 from C to B
Move Disk 1 from A to B
Move Disk 4 from A to C
Move Disk 1 from B to C
Move Disk 2 from B to A
Move Disk 1 from C to A
Move Disk 3 from B to C
Move Disk 1 from A to B
Move Disk 2 from A to C
Move Disk 1 from B to C

~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 3/Source-Code
```

Code:

```
/*  
Program Name: Towers of Hanoi Using Recursion  
Program Description: Takes the input {number of disks} and solves the Towers  
of Hanoi Problem using recursive approach  
Program Creation Time: Aug 20st 4:50  
Program Creator: Ashwath Raj | AP25110010712  
*/  
  
#include<iostream>  
using namespace std;  
  
void towerOfHanoi(int n,char source,char aux,char dest);  
  
int main(void){  
cout<<"======"<<endl;  
cout<<"TOWERS OF HANOI USING RECURSION"<<endl;  
cout<<"======"<<endl;  
  
int n;  
  
cout<<"Enter Number of Disks: ";  
cin>>n;  
  
if(n<=0){  
cout<<"Invalid Number of Disks!!"<<endl;  
return 0;  
}  
  
towerOfHanoi(n,'A','B','C');  
  
return 0;  
}  
  
void towerOfHanoi(int n,char source,char aux,char dest){  
if(n==1){  
cout<<"Move Disk 1 from "<<source<<" to "<<dest<<endl;  
return;  
}  
  
towerOfHanoi(n-1,source,dest,aux);  
cout<<"Move Disk "<<n<<" from "<<source<<" to "<<dest<<endl;  
towerOfHanoi(n-1,aux,source,dest);  
}
```

Complexity

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$

Method b — Implement Without Recursion

Problem

Implement the Towers of Hanoi problem without using recursion.

Method

The recursive solution is converted into a non-recursive approach by explicitly managing the required disk movements.

Output:

```
Problems Output Debug Console Terminal Ports
~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 3/Source-Code cd "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3 TowerOfHanoiLLstackandqueue/Source-Code/" && g++ 1.bTowersofHanoiProblemIterative.cpp -o 1.bTowersofHanoiProblemIterative && "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3 TowerOfHanoiLLstackandqueue/Source-Code/"1.bTowersofHanoiProblemIterative

=====
TOWERS OF HANOI WITHOUT RECURSION
=====
Enter Number of Disks: 4
Move Disk 1 from A to B
Move Disk 2 from A to C
Move Disk 1 from B to C
Move Disk 3 from A to B
Move Disk 1 from C to A
Move Disk 2 from C to B
Move Disk 1 from A to B
Move Disk 4 from A to C
Move Disk 1 from B to C
Move Disk 2 from B to A
Move Disk 1 from C to A
Move Disk 3 from B to C
Move Disk 1 from A to B
Move Disk 2 from A to C
Move Disk 1 from B to C

~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 3/Source-Code
```

```
Problems Output Debug Console Terminal Ports
~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 3/Source-Code cd "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3 TowerOfHanoiLLstackandqueue/Source-Code/" && g++ 1.bTowersofHanoiProblemIterative.cpp -o 1.bTowersofHanoiProblemIterative && "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3 TowerOfHanoiLLstackandqueue/Source-Code/"1.bTowersofHanoiProblemIterative

=====
TOWERS OF HANOI WITHOUT RECURSION
=====
Enter Number of Disks: -3
Invalid Number of Disks!!

~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 3/Source-Code
```

Code:

```
/*  
Program Name: Towers of Hanoi Without Recursion  
Program Description: Takes the input {number of disks} and solves the Towers  
of Hanoi Problem using iterative approach  
Program Creation Time: Aug 20st 03:15  
Program Creator: Ashwath Raj | AP25110010712  
*/  
  
#include<iostream>  
#include<stack>  
using namespace std;  
  
void moveDisk(stack<int>& a,stack<int>& b,char x,char y){  
int d1=a.empty()?0:a.top();  
int d2=b.empty()?0:b.top();  
  
if(d1==0){  
a.push(d2);  
b.pop();  
cout<<"Move Disk "<<d2<<" from "<<y<<" to "<<x<<endl;  
}  
else if(d2==0){  
b.push(d1);  
a.pop();  
cout<<"Move Disk "<<d1<<" from "<<x<<" to "<<y<<endl;  
}  
else if(d1<d2){  
b.push(d1);  
a.pop();  
cout<<"Move Disk "<<d1<<" from "<<x<<" to "<<y<<endl;  
}  
else{  
a.push(d2);  
b.pop();  
cout<<"Move Disk "<<d2<<" from "<<y<<" to "<<x<<endl;  
}  
}  
  
int main(void){  
cout<<"======"<<endl;  
cout<<"TOWERS OF HANOI WITHOUT RECURSION"<<endl;  
cout<<"======"<<endl;  
  
int n;  
cout<<"Enter Number of Disks: ";  
cin>>n;  
  
if(n<=0){  
cout<<"Invalid Number of Disks!!"<<endl;
```

```

return 0;
}

stack<int> source,aux,dest;

for(int i=n;i>=1;i--)
source.push(i);

int moves=(1<<n)-1;

for(int i=1;i<=moves;i++){
if(n%2==1){
if(i%3==1)
moveDisk(source,dest,'A','C');
else if(i%3==2)
moveDisk(source,aux,'A','B');
else
moveDisk(aux,dest,'B','C');
}
else{
if(i%3==1)
moveDisk(source,aux,'A','B');
else if(i%3==2)
moveDisk(source,dest,'A','C');
else
moveDisk(aux,dest,'B','C');
}
}

return 0;
}

```

Complexity

Time Complexity: $O(2^n)$

Space Complexity: Depends on the implementation

2. Stack Operations Using Linked List

Problem

Implement stack operations using a linked list.

Method

A linked list is used to implement the stack following the **LIFO (Last In, First Out)** principle.

The following operations are performed:

- Push
- Pop
- Display

Output:



```
~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 3/Source-Code cd "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3_ TowerOfHanoiLLstackandqueue/Source-Code/" && g++ 2.StackoperationsusingLinkedList.cpp -o 2.StackoperationsusingLinkedList && "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3_ TowerOfHanoiLLstackandqueue/Source-Code/"2.StackoperationsusingLinkedList
Enter Number of Elements: 4
Enter Element 1: 3
Enter Element 2: 7
Enter Element 3: 8
Enter Element 4: 3
Stack After Push: Stack: 3 8 7 3
Popped Element: 3
Stack After Pop: Stack: 8 7 3
~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 3/Source-Code
```

Code:

```
/*
Program Name: Stack Implementation Using Linked List
Program Description: Implements Stack operations using a Linked List
following the LIFO principle
Program Creation Time: Aug 21st 10:00
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
using namespace std;

struct ListNode{
int val;
ListNode *next;

ListNode():val(0),next(nullptr){}
ListNode(int x):val(x),next(nullptr){}
ListNode(int x,ListNode *next):val(x),next(next){}
};
```

```

class Stack{
    ListNode *top;

public:
    Stack(){
        top=nullptr;
    }

    void push(int val){
        ListNode *temp=new ListNode(val);
        temp->next=top;
        top=temp;
    }

    void pop(){
        if(top==nullptr){
            cout<<"Stack is empty!!"<<endl;
            return;
        }

        ListNode *temp=top;
        cout<<"Popped Element: "<<temp->val<<endl;
        top=temp->next;
        delete temp;
    }

    void display(){
        ListNode *temp=top;

        cout<<"Stack: ";
        while(temp!=nullptr){
            cout<<temp->val<<" ";
            temp=temp->next;
        }
        cout<<endl;
    }
};

int main(void){
    Stack s;
    int n,val;

    cout<<"Enter Number of Elements: ";
    cin>>n;

    for(int i=0;i<n;i++){
        cout<<"Enter Element "<<i+1<<": ";
        cin>>val;
        s.push(val);
    }
}

```

```
cout<<"Stack After Push: ";  
s.display();
```

```
s.pop();
```

```
cout<<"Stack After Pop: ";  
s.display();
```

```
return 0;  
}
```

Complexity

Push: $O(1)$

Pop: $O(1)$

Display: $O(n)$

Space Complexity: $O(n)$

3. Queue Operations Using Linked List

Problem

Implement queue operations using a linked list.

Method

A linked list is used to implement the queue following the **FIFO (First In, First Out)** principle.

The following operations are performed:

- Enqueue
- Dequeue
- Display

Output:



```
Problems Output Debug Console Terminal Ports
~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 3/Source-Code cd "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3_ TowerOfHannoilLstackandqueue/Source-Code/" && g++ 3.QueueoperationsusingLinkedList.cpp -o 3.QueueoperationsusingLinkedList && "/home/ashwath-raj/Acadimics/3.Semister3/Coding/Assignments DAA/Designing and Analysisi of Algorithm/CSE-204-DAA-assignments-/LAB 3_ TowerOfHannoilLstackandqueue/Source-Code/"3.QueueoperationsusingLinkedList
Enter Number of Elements: 3
Enter Element 1: 5
Enter Element 2: 66
Enter Element 3: -99
Queue After Enqueue: Queue: 5 66 -99
Dequeued Element: 5
Queue After Dequeue: Queue: 66 -99
~/A/3/Co/Assignments D/D/CSE-204-DAA-assignments-/LAB 3/Source-Code
```

Code:

```
/*
Program Name: Queue Implementation Using Linked List
Program Description: Implements Queue operations using a Linked List
following the FIFO principle
Program Creation Time: Aug 21st 12:05
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
using namespace std;

struct ListNode{
int val;
ListNode *next;

ListNode():val(0),next(nullptr){}
ListNode(int x):val(x),next(nullptr){}
ListNode(int x,ListNode *next):val(x),next(next){}
};

class Queue{
ListNode *front,*rear;
```

```
public:
Queue(){
front=rear=nullptr;
}
```

```
void enqueue(int val){
ListNode *temp=new ListNode(val);
```

```
if(rear==nullptr){
front=rear=temp;
return;
}
```

```
rear->next=temp;
rear=temp;
}
```

```
void dequeue(){
if(front==nullptr){
cout<<"Queue is empty!!"<<endl;
return;
}
```

```
ListNode *temp=front;
cout<<"Dequeued Element: "<<front->val<<endl;
front=front->next;
```

```
if(front==nullptr)
rear=nullptr;
```

```
delete temp;
}
```

```
void display(){
ListNode *temp=front;
```

```
cout<<"Queue: ";
while(temp!=nullptr){
cout<<temp->val<<" ";
temp=temp->next;
}
cout<<endl;
}
};
```

```
int main(void){
Queue q;
int n,val;
```

```
cout<<"Enter Number of Elements: ";
```

```
cin>>n;

for(int i=0;i<n;i++){
cout<<"Enter Element "<<i+1<<" ";
cin>>val;
q.enqueue(val);
}

cout<<"Queue After Enqueue: ";
q.display();

q.dequeue();

cout<<"Queue After Dequeue: ";
q.display();

return 0;
}
```

Complexity

Enqueue: $O(1)$

Dequeue: $O(1)$

Display: $O(n)$

Space Complexity: $O(n)$